ScanGo

Aplicación de autocompra para tendas físicas

https://github.com/albarf1/ScanGo

Nome Alumno/a: Alba Rodríguez Fernández

Curso: ***2º DAM***

Módulo: ***Proxecto Final Ciclo***

Contido

1. Introdución 3
2. Obxectivos 3
3. Situación previa 3
4. Tecnoloxías empregadas 4
    Frontend 4
    Backend 5
    Base de datos 5
    Control de versións 5
5. Solución proposta 6
6. Presuposto 6
    6.1 Hardware 6
    6.2 Software 7
        6.2.1 Xustificación técnica da elección 7
    6.3 Recursos humanos 7
    6.4 Resumo do custo total 8
7. Planificación do proxecto 8
    7.1 Diagrama de Gantt 8
    7.2 Modelo de desenvolvemento de software 9
8. Análise do proxecto 10
    8.1 Diagrama de casos de uso 10
        8.1.1 Táboas descriptivas de cada caso 11
    8.2 Universo de discurso da base de datos 22
    8.3 Diagrama Entidade-Relación 23
9. Deseño do proxecto 24
    9.1 Modelo relacional da base de datos 24
    9.2 Diagrama de clases 25
    9.3 Diagramas de secuencia de cada caso de uso 26
    9.4 Diagrama de despregamento 32
        9.4.1 Nodos e contornas de execución 32
        9.4.2 Protocolos e comunicacións 33
        9.4.3 Seguridade 33
10.Mockups da interface. 34
11. Especificación de alcance do sistema 37
    11.1 Funcionalidades integradas na fase actual 37
    11.2 Funcionalidades diferidas (Iteracións posteriores) 38
            A. Autenticación baseada en JWT (CU01, CU02, CU10) 38
            B. Persistencia e Edición do Carriño (CU05) 38
            C. Proceso de Pago e Venda (CU06, CU07) 38
            D. Xestión Administrativa (CU08, CU09) 39
            E. Validación de Saída (CU11) 39
    11.3 Xustificación de decisións técnicas 39
    11.4 Límites do sistema 39
    11.5 Funcionalidades fora do alcance nesta fase 40
        11.5.1 Responsabilidades 40

1. Introdución

ScanGo é unha aplicación móbil deseñada para facilitar o proceso de compra en tendas físicas mediante un sistema de autocompra dixital.

O cliente pode escanear os produtos co seu teléfono móbil mentres percorre a tenda, engadilos a un carriño virtual e consultar o importe total en tempo real sen necesidade de pasar por unha caixa tradicional.

A aplicación está orientada tanto a pequenos comercios como a supermercados que desexen modernizar o seu sistema de venda, reducir colas e mellorar a experiencia do cliente. Ademais, permite optimizar o tempo de compra e ofrecer un servizo máis cómodo e eficiente.

O sistema tamén inclúe unha plataforma de administración que permite xestionar produtos, controlar o stock e consultar información básica das vendas.

2. Obxectivos

Obxectivos iniciais

- Desenvolver unha aplicación móbil intuitiva que permita escanear produtos mediante códigos QR.
- Permitir aos clientes xestionar un carriño virtual e consultar o custo total da compra en tempo real.
- Deseñar un sistema backend robusto que xestione produtos, stock e transaccións de forma eficiente.
- Crear unha base de datos estruturada para almacenar información de produtos, usuarios e vendas.
- Garantir a comunicación segura entre a aplicación móbil e o servidor mediante API REST.
- Implementar un sistema de autenticación básica de usuarios.
- Facilitar a xestión de produtos e stock por parte dos administradores.
- Reducir os tempos de espera nas tendas físicas.
- Mellorar a experiencia de usuario mediante unha interface sinxela e clara.

3. Situación previa

Nas tendas físicas tradicionais, os clientes deben esperar en longas colas para realizar o pago dos seus produtos, especialmente en horas punta ou en períodos de gran afluencia de xente. Esta situación provoca unha experiencia de compra negativa e perdas de tempo innecesarias.

Ademais, as colas poden xerar perdas económicas para os comercios, xa que algúns clientes abandonan as súas compras por falta de tempo ou impaciencia.

Nos últimos anos, o comercio electrónico medrou considerablemente debido á súa rapidez e comodidade, polo que as tendas físicas necesitan modernizarse para poder competir e ofrecer servizos máis áxiles.

Aínda que existen solucións de autocompra en grandes superficies, moitas pequenas e medianas tendas non dispoñen de sistemas accesibles debido ao seu elevado custo ou á complexidade de implantación.

ScanGo nace como unha solución tecnolóxica accesible que permite dixitalizar o proceso de compra sen necesidade de grandes infraestruturas, facilitando a modernización dos comercios tradicionais.

4. Tecnoloxías empregadas

Frontend

Flutter

Google desenvolveu Flutter, unha ferramenta gratuíta que permite crear aplicacións móbiles para Android e iOS usando un único código. Características principais:

- Permite crear unha soa aplicación que funciona en varios dispositivos.
- As aplicacións son rápidas e funcionan de forma fluída.
- Usa compoñentes visuais chamados widgets para crear interfaces atractivas.
- Permite ver os cambios ao instante mentres se programa grazas á función Hot Reload.
- Ten moita documentación e unha gran comunidade de desenvolvedores.

Flutter é ideal para este proxecto porque permite crear unha aplicación moderna e visual sen necesidade de programar dúas versións distintas.

Dart

Dart é a linguaxe de programación creada por Google que se emprega para desenvolver aplicacións con Flutter. Características principais:

- Fácil de aprender para persoas que xa programaron antes.
- Permite organizar ben o código usando programación orientada a obxectos.
- Funciona perfectamente xunto con Flutter.
- Permite realizar varias tarefas ao mesmo tempo (por exemplo, cargar datos de internet).
- As aplicacións feitas con Dart funcionan de maneira rápida e eficiente.

Backend

FastAPI

FastAPI é unha ferramenta que permite crear o servidor da aplicación usando Python. O servidor é o encargado de xestionar os datos, os usuarios e as comunicacións coa aplicación móbil. Características principais:

- Moi rápido e eficiente.
- Permite crear servizos web de forma ordenada.
- Xera documentación automática para probar a aplicación.
- Comproba automaticamente que os datos enviados sexan correctos.
- Utiliza tecnoloxías estándar de internet.

FastAPI permite que a aplicación móbil se comunique co servidor de forma segura e rápida.

Base de datos

PostgreSQL

PostgreSQL é un sistema xestor de bases de datos relacional de código aberto, amplamente utilizado en contornos profesionais. Características principais:

- Garda os datos de forma segura.
- Permite almacenar moita información.
- Organiza os datos en táboas relacionadas entre si.
- É moi utilizado por empresas e aplicacións profesionais.

Empregarase para gardar información de produtos, usuarios e compras.

Control de versións

Git

Git é unha ferramenta que permite gardar o historial de cambios realizados no código dun proxecto. Características principais:

- Garda todas as modificacións feitas no proxecto.
- Permite traballar varias persoas á vez sen perder información.
- Posibilita volver a versións anteriores se hai erros.
- Permite probar novas melloras sen afectar ao proxecto principal.

GitHub

GitHub é unha plataforma web onde se gardan proxectos programados con Git. Características principais:

- Garda copias de seguridade do proxecto en internet.

- Permite traballar en equipo facilmente.
- Axuda a organizar tarefas e melloras do proxecto.
- Úsase amplamente en proxectos profesionais.

5. Solución proposta

Proponse o desenvolvemento dun sistema de autocompra no que a aplicación do cliente se comunica cun servidor central que xestiona toda a información. O sistema estará composto por:

- Unha aplicación móbil para que os clientes poidan escanear e mercar produtos.
- Un servidor que procesa os datos e controla o funcionamento da aplicación.
- Unha base de datos que almacena toda a información dos produtos, usuarios e compras.
- Un panel de administración para que a tenda poida xestionar produtos e controlar o stock.

Este deseño permite separar a parte visual que usa o cliente da parte interna que xestiona os datos, facilitando así o mantemento da aplicación e permitindo engadir melloras no futuro. A diferenza doutras solucións comerciais, ScanGo será:

- Adaptable a pequenas e medianas tendas.
- Personalizable segundo as necesidades de cada negocio.
- Máis económico ao ser unha solución propia.
- Doado de integrar cos sistemas que xa emprega a tenda.

Cómpre aclarar que o alcance de ScanGo céntrase na dixitalización do proceso de escaneo e pago polo propio cliente. Para garantir a seguridade física e evitar erros na cantidade de produtos retirados, o sistema está deseñado para integrarse con infraestruturas externas do establecemento. Dependendo do tamaño da tenda, esta validación pode realizarse mediante o pesaxe dos artigos nunha báscula de control que xere un código QR de saída, ou ben mediante unha comprobación manual por parte dun operario que valide a compra escanando un código temporal no dispositivo do cliente.

6. Presuposto

O presuposto do proxecto inclúe tanto o equipamento e software necesario para o desenvolvemento como os recursos humanos estimados. O obxectivo é ter unha visión completa do custo do proxecto.

6.1 Hardware

- Equipamento de desenvolvemento: PC portátil (Procesador i7, 16 GB RAM) — 900 €. Utilizado para programar, probar e compilar a aplicación.
- Dispositivo de probas: Smartphone Android de gama media — 200 €. Usado para testar a aplicación en condicións reais de uso.

6.2 Software

Para o despregamento en produción de **ScanGo**, seleccionouse o provedor de servizos na nube **Render.com**, debido á súa compatibilidade nativa con aplicacións Python (FastAPI) e bases de datos PostgreSQL, así como pola súa facilidade de integración continua con GitHub.

Servizo	Descrición técnica	Custo mensual	Custo anual
Backend (FastAPI)	1.100	7,00 €	84,00€
Base de Datos (PostgreSQL)	186	7,00 €	84,00€
Dominio Personalizado	Rexistro de dominio .com ou .es con redirección SSL para a API do sistema.	1,50 €	18,00 €
Total		15,50 €	186,00 €

6.2.1 Xustificación técnica da elección

A elección de Render.com como plataforma de hospedaxe xustifícase polas seguintes características técnicas necesarias para o proxecto:

- **Integración Continua (CI/CD)**: Cada vez que se realiza un *commit* na rama principal de GitHub, o servidor actualízase automaticamente.
- **Seguridade**: Provisión automática de certificados SSL para garantir que a comunicación entre a App Flutter e o servidor FastAPI sexa cifrada.
- **Escalabilidade**: Permite aumentar os recursos de CPU e RAM de forma sinxela se o volume de usuarios de ScanGo aumenta segundo o previsto.
- **Xestión de BD**: PostgreSQL xestionado asegura a dispoñibilidade dos datos e a integridade referencial sen necesidade de administración manual do servidor de base de datos.

6.3 Recursos humanos

- Analista-Programador: 300 horas de traballo a un prezo de mercado de 25 €/hora — 7.500 €. Inclúe o deseño, desenvolvemento, probas e documentación do proxecto.

6.4 Resumo do custo total

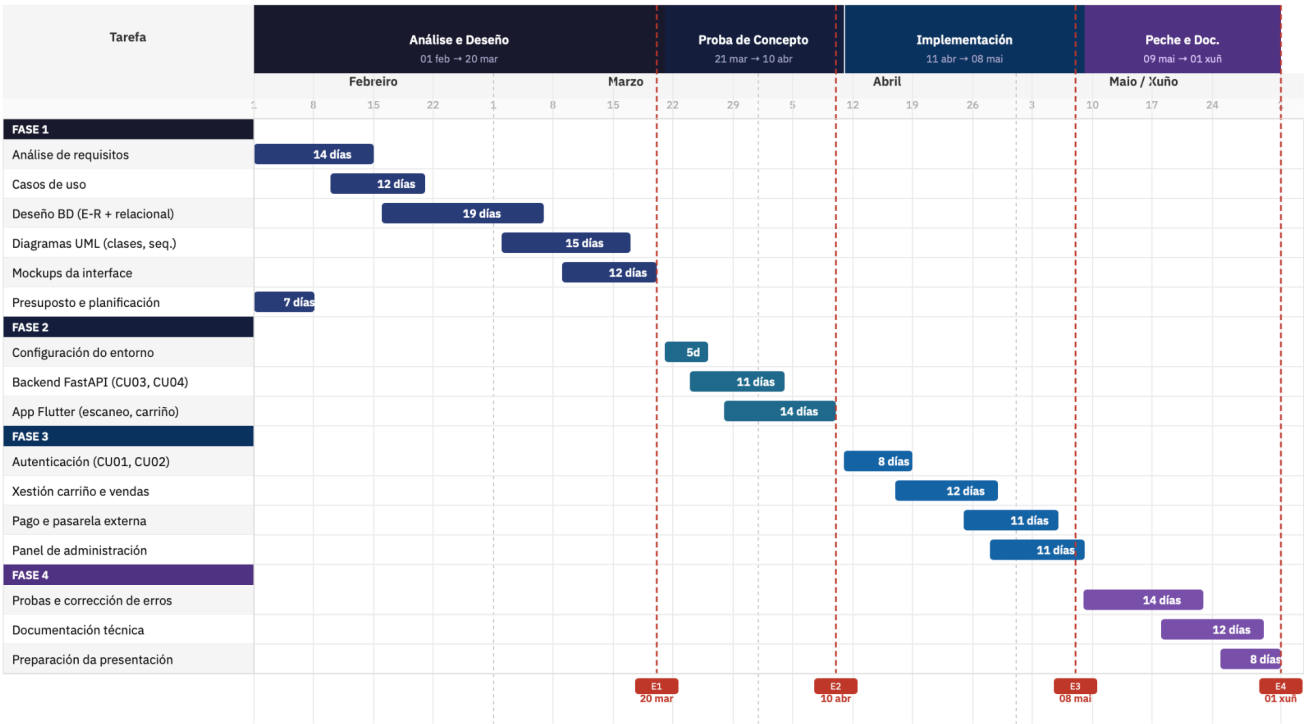
Concepto	Custo (€)
Hardware	1.100
Software	186
Recursos Humanos	7.500
Total estimado	8.786

7. Planificación do proxecto

A planificación do proxecto establece as fases de traballo, as tarefas asociadas e os prazos de entrega. Estrutúrase en catro fases iterativas seguindo o modelo de desenvolvemento áxil adoptado para ScanGo.

7.1 Diagrama de Gantt

O diagrama de Gantt representa visualmente a distribución temporal das tarefas ao longo do período de desenvolvemento, comprendido entre o 1 de febreiro e o 1 de xuño de 2026. Cada fase aparece identificada cunha cor diferente e as datas de entrega marcadas con liñas verticais.



7.2 Modelo de desenvolvemento de software

Para o desenvolvemento do proxecto ScanGo decidiuse empregar un modelo de desenvolvemento Áxil Iterativo e Incremental, inspirado en metodoloxías como Scrum, aínda que adaptado ás características dun proxecto individual.

A diferenza de modelos máis ríxidos como o modelo en cascada, nos que cada fase debe completarse antes de pasar á seguinte, o enfoque áxil permite avanzar mediante pequenas iteracións funcionais do produto, facilitando a detección temperá de erros e a adaptación ante posibles dificultades técnicas.

Xustificación da elección

O principal motivo para escoller este modelo é a flexibilidade que ofrece. Ao tratarse dun proxecto individual, no que a mesma persoa realiza tarefas de análise, deseño e desenvolvemento, resulta fundamental dispoñer dun sistema que permita reorganizar o traballo en función do progreso real.

Ademais, o calendario de entregas establecido polo departamento favorece este enfoque, xa que cada fase require a presentación de resultados funcionais parciais. Isto encaixa directamente coa filosofía iterativa, onde cada ciclo produce unha mellora incremental do sistema.

Organización das iteracións

Fase 1 — Análise e Deseño (ata o 20 de marzo):

Defínense os requisitos do sistema, realízase o deseño da base de datos en PostgreSQL, elabóranse os diagramas UML e deséñanse os mockups da interface. Tamén se completa o presuposto e a planificación temporal.

Fase 2 — Proba de Concepto (ata o 10 de abril):

Desenvólvese un caso de uso completo que permita validar o funcionamento básico do sistema, como o escaneo de produtos e a súa engadida ao carriño virtual. Esta fase permite comprobar a correcta comunicación entre o frontend móbil e o backend.

Fase 3 — Implementación Completa (ata o 8 de maio):

Engádense o resto de funcionalidades: rexistro e autenticación de usuarios, simulación do pago final e panel de administración para a xestión de produtos e control de stock.

Fase 4 — Peche e Documentación (ata o 1 de xuño):

Realízanse probas finais de estabilidade, corríxense erros non críticos e complétase a documentación técnica e de usuario. Prepárase a presentación para a defensa do proxecto.

Xestión do código e control de versións

Empregarase Git como sistema de control de versións, utilizando GitHub como repositorio remoto. Realizaranse commits frecuentes, cun mínimo de dúas actualizacións diarias durante as fases activas de desenvolvemento, o que permite manter un historial detallado do progreso, recuperar versións anteriores en caso de erros e organizar o traballo por funcionalidades.

8. Análise do proxecto

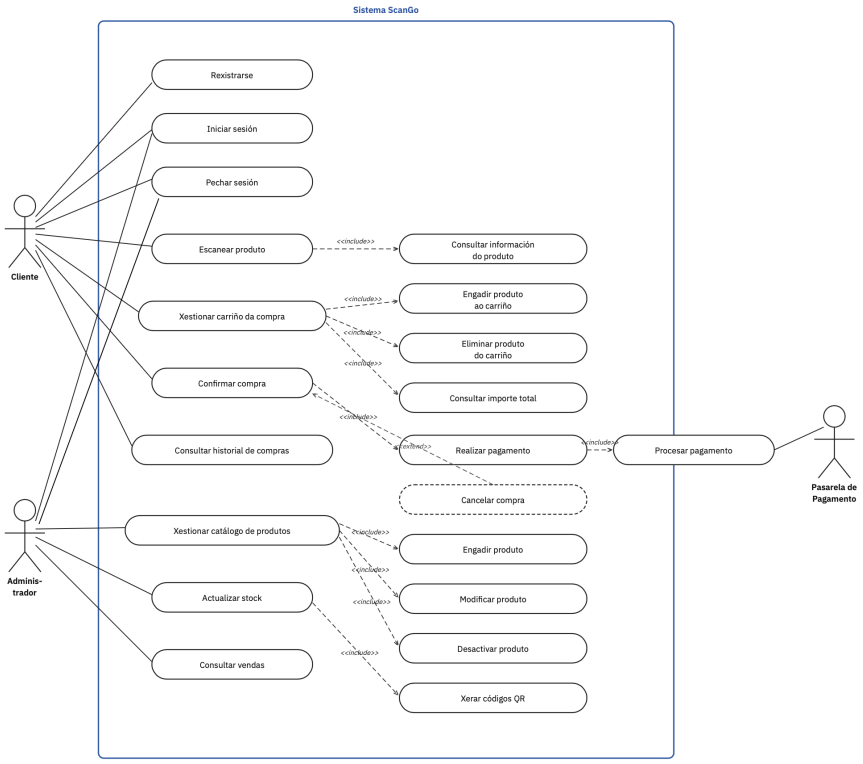
8.1 Diagrama de casos de uso

Co obxectivo de representar de maneira clara as funcionalidades do sistema e a interacción dos usuarios coa aplicación, elaborouse o diagrama de casos de uso de ScanGo empregando a notación estándar UML. Este diagrama permite identificar os distintos actores que interveñen no sistema e as accións que cada un deles pode realizar.

Os actores principais definidos son o Cliente, o Administrador e a Pasarela de Pagamento como sistema externo. O Cliente interactúa coa aplicación móbil para rexistrarse, iniciar sesión, escanear produtos, xestionar o carriño e finalizar o proceso de compra. O Administrador accede ao sistema mediante un panel de control que lle permite xestionar o catálogo de produtos, actualizar o stock dispoñible e consultar o rexistro de vendas realizadas.

A Pasarela de Pagamento actúa como entidade externa encargada de procesar as transaccións económicas derivadas das compras realizadas polos clientes. A súa participación resulta fundamental para completar o proceso de pagamento de maneira segura.

O diagrama tamén representa relacións de inclusión («include») entre casos de uso, permitindo modelar funcionalidades que forman parte doutras operacións máis complexas, conseguindo así unha representación estruturada e comprensible do comportamento funcional do sistema.



8.1.1 Táboas descriptivas de cada caso

CU01	Rexistrarse		
Descripción	O cliente pode crear unha conta en ScanGo para xestionar as súas compras e o seu perfil. O sistema comproba que os datos son correctos e, unha vez completado o rexistro, habilita o acceso á aplicación.		
Actores	Cliente		
Pre condiciones	1. Cliente non debe ter conta existente no sistema. 2. O Cliente dispón de acceso á internet. 3. A aplicación está instalada no dispositivo móbil.		
Post condiciones	1. Créase a conta do Cliente na base de datos. 2. O Cliente pode autenticarse no sistema e acceder ás súas funcionalidades. 3. O cliente recibe confirmación do rexistro.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Cliente selecciona "Rexistrarse"	Sistema mostra formulario de rexistro
	2	Cliente introduce datos persoais e contrasinal	Sistema valida os datos e crea a conta
			• 2.1 Se o correo xa existe, o sistema amosa unha mensaxe de erro e suxire iniciar sesión • 2.2 Se os datos están incompletos ou incorrectos, o sistema amosa erro e solicita corrección
	3	Cliente confirma o rexistro	Sistema confirma que a conta foi creada e habilita acceso
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Cliente intenta rexistrarse e falla a conexión	Sistema mostra mensaxe de erro e permite reintento
	q	O servidor non responde	Sistema ofrece rexistro manual ou intentar máis tarde
Rendimiento	O sistema debe crear a conta en menos de 2 segundos desde o envío do formulario.		
Frecuencia	Estímase unha media de 50 rexistros diarios.		
Importancia	Vital		
Urgencia	Inmediata		
Comentarios	1. A contrasinal debe cumprir normas de seguridade (mínimo 8 caracteres, combinación de letras e números). 2. O sistema debe comprobar duplicidade de datos antes de crear a conta. 3. Validar formato de correo electrónico e outros campos obrigatorios antes de enviar. 4. Considerar a opción de enviar correo de confirmación de rexistro.		

	5. Posibilidade de rexistrarse mediante sistemas externos (Google, Facebook) para mellorar a experiencia do usuario.Como melloras futuras

CU02	**Iniciar sesión**		
Descripción	Permite que o usuario (Cliente ou Administrador) inicie sesión no sistema para acceder ás funcionalidades correspondentes ao seu rol.		
Actores	Cliente, Administrador		
Pre condiciones	1. O usuario debe ter conta activa. 2. O usuario dispón de acceso á internet. 3. A aplicación está instalada no dispositivo.		
Post condiciones	1. O usuario queda autenticado e pode realizar as funcións correspondentes ao seu rol. 2. O sistema garda a sesión activa.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Usuario selecciona "Iniciar sesión"	Sistema mostra formulario de login
	2	Usuario introduce email e contrasinal	Sistema valida credenciais
			● 2.1Se as credenciais son incorrectas, o sistema amosa erro e permite reintento ● 2.2 Se o usuario esqueceu a contrasinal, o sistema ofrece opción de recuperar contrasinal
	3	Usuario queda autenticado	Sistema redirixe ao panel correspondente segundo o rol
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	En caso de fallo de conexión ou erro do servidor	Sistema amosa aviso de erro e permite reintento
	q	En caso de que o usuario introduza datos incorrectos repetidamente	Sistema bloquea temporalmente acceso e avisa ao usuario
Rendimiento	O sistema debe validar as credenciais en menos de 2 segundos desde o envío do formulario		
Frecuencia	Estímase unha media de 200 accesos diarios.		
Importancia	Vital		
Urgencia	Inmediata		
Comentarios	1. Limitar intentos de login incorrectos para evitar ataques de forza bruta. 2. Mostrar mensaxes claras e amigables en caso de erro. 3. Considerar autenticación multifactor para usuarios administrativos. 4. Posibilidade de autenticación mediante sistemas externos.		

CU03	**Escanear produto**		
Descripción	O cliente pode escanear o código QR dun produto utilizando a cámara do seu dispositivo móbil para obter a información do produto e engadilo ao carrito.		
Actores	Cliente		
Pre condiciones	1. O cliente debe ter iniciado sesión na aplicación. 2. O dispositivo ten permisos de cámara activados. 3. O produto ten un código QR válido e lexible.		
Post condiciones	1. O produto identifícase correctamente no sistema. 2. Móstrase a información do produto (nome, prezo, stock). 3. O cliente pode decidir engadir o produto ao carrito.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Cliente abre a cámara desde a aplicación	Sistema activa o lector de códigos QR
	2	Cliente enfoca o código QR do produto	Sistema le o código e consulta a base de datos
			• 2.1 Se o código QR non é válido, o sistema amosa mensaxe de erro e permite intentar de novo • 2.2 Se o produto non existe, o sistema amosa aviso de produto non encontrado
	3	Sistema obtén información do produto	Sistema mostra a información do produto (nome, prezo, stock)
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Non hai conexión a internet	Sistema garda o código en caché e sincroniza cando haxa conexión
	q	A cámara non funciona correctamente	Sistema amosa erro e suxire revisar permisos
Rendimiento	O sistema debe identificar o produto en menos de 1 segundo desde o escaneo.		
Frecuencia	Estímase unha media de 500 escaneos diarios.		
Importancia	Vital		
Urgencia	Inmediata		
Comentarios	1. O lector QR debe funcionar en condicións de pouca luz. 2. Validar que o código QR corresponde a un produto activo na base de datos. 3. Mostrar información clara do produto despois do escaneo. 4. Permitir escaneo múltiple sen pechar a cámara. 5. Considerar o uso do flash automático en ambientes escuros.		

CU04	**Engadir produto ao carrito**		
Descripción	O cliente pode engadir un produto escaneado ao seu carrito virtual para incluílo na súa compra.		
Actores	Cliente		
Pre condiciones	1. O produto foi escaneado correctamente. 2. O cliente ten unha sesión activa. 3. Hai stock dispoñible do produto.		
Post condiciones	1. O produto engádese ao carrito do cliente. 2. Actualízase o importe total da compra. 3. O stock resérvase temporalmente.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Cliente visualiza a información do produto escaneado	Sistema mostra detalles do produto
	2	Cliente preme o botón "Engadir ao carrito"	Sistema verifica que hai stock dispoñible
			• 2.1 Se o cliente xa ten ese produto no carrito, o sistema incrementa a cantidade en lugar de crear un duplicado. • 2.2 Se non hai stock suficiente, o sistema amosa alerta e non permite engadir o produto.
	3	Sistema engade o produto ao carrito	Sistema actualiza o importe total da compra en tempo real
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Non hai stock suficiente	Sistema mostra alerta e non permite engadir o produto
	q	Erro de conexión co servidor	Sistema garda localmente e sincroniza despois
Rendimiento	O sistema debe engadir o produto en menos de 1 segundo		
Frecuencia	Estímase unha media de 500 rexistros diarios.		
Importancia	Vital		
Urgencia	Inmediata		
Comentarios	1. Mostrar cantidade actual no carrito despois de engadir. 2. Permitir modificar a cantidade antes de finalizar. 3. Actualizar o importe total en tempo real. 4. Validar stock antes de confirmar a engadida. 5. Permitir engadir múltiples unidades do mesmo produto.		

CU05	Ver carrito de compra		
Descripción	O cliente pode consultar os produtos que engadiu ao seu carrito virtual, ver as cantidades e o importe total acumulado.		
Actores	Cliente		
Pre condiciones	1. O cliente ten produtos no carrito. 2. O cliente iniciou sesión na aplicación		
Post condiciones	1. O cliente visualiza todos os produtos do carrito. 2. O cliente pode modificar cantidades ou eliminar produtos. 3. O importe total está actualizado.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Cliente accede á vista do carrito	Sistema carga os produtos do carrito
	2	Sistema procesa o contigo do carrito	Sistema mostra nome, prezo, cantidade e subtotal de cada produto
			• 2.1 Se o carrito está baleiro, o sistema amosa mensaxe e suxire seguir comprando
	3	Sistema calcula importe total	Sistema mostra o total actualizado da compra
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Erro ao cargar os datos	Sistema amosa mensaxe de erro e permite reintentar
	q	Algún produto xa non ten stock	Sistema avisa e permite eliminar o produto do carrito
Rendimiento	O sistema debe cargar o carrito en menos de 2 segundos.		
Frecuencia	Estímase unha media de 250 consultas diarias.		
Importancia	Vital		
Urgencia	Inmediata		
Comentarios	1. Mostrar imaxe do produto se está dispoñible. 2. Permitir modificar cantidades desde o carrito. 3. Actualizar prezos se cambiaron desde que se engadiu o produto. 4. Mostrar produtos esgotados de forma diferenciada. 5. Permitir gardar o carrito para máis tarde.		

CU06	Finalizar compra		
Descripción	O cliente confirma os produtos do carrito e prepárase para realizar o pago. Nesta fase vérifícase o stock e resérvase temporalmente, pero o stock non se desconta ata completar o pago (CU07).		
Actores	Cliente		
Pre condiciones	1. O cliente ten produtos no carrito. 2. O cliente iniciou sesión. 3. Todos os produtos teñen stock dispoñible.		
Post condiciones	1. A compra confírmase e prepárase para o pago. 2. O stock resérvase temporalmente (non se desconta ata completar o pago en CU07). 3. Xérase un identificador provisional de venda.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Cliente accede á vista do carrito	Sistema mostra resumo da compra
	2	Cliente revisa produtos e importe total	Sistema permite modificar ou eliminar produtos
	3	Cliente preme "Finalizar compra"	Sistema verifica que todos os produtos teñen stock
			• 3.1 Se algún produto quedou sen stock, o sistema avisa e elimina ese produto do carrito
	4	Sistema reserva stock temporalmente	Sistema redirixe ao cliente á pantalla de pago
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Erro ao verificar stock	Sistema amosa erro e permite reintentar
	q	O cliente cancela o proceso	Sistema libera o stock reservado e mantén o carrito
Rendimiento	O proceso debe completarse en menos de 3 segundos		
Frecuencia	Estímase unha media de 100 compras diarias.		
Importancia	Vital		
Urgencia	Inmediata		
Comentarios	1. Mostrar resumo claro antes de confirmar. 2. Reservar stock por tempo limitado (ex: 10 minutos) para evitar bloqueos indefinidos. 3. Xerar comprobante temporal da compra. 4. Permitir voltar atrás antes de confirmar o pago. 5. O desconto definitivo do stock realízase en CU07 tras confirmar o pago.		

CU07	Realizar pago		
Descripción	O cliente completa a transacción económica da súa compra utilizando un método de pago dispoñible.		
Actores	Cliente, Sistema de pago		
Pre condiciones	1. A compra foi finalizada correctamente (CU06). 2. O cliente ten seleccionado un método de pago. 3. O stock está reservado temporalmente.		
Post condiciones	1. O pago rexístrase como completado. 2. Descóntase o stock dos produtos da base de datos. 3. Xérase o ticket de compra. 4. O carrito limpase.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Cliente selecciona método de pago	Sistema mostra opcións dispoñibles
	2	Cliente introduce datos de pago	Sistema procesa a transacción coa pasarela
			• 2.1 Se o pago falla, o sistema permite intentar con outro método
	3	Pasarela confirma pago	Sistema rexistra a venda na base de datos e desconta o stock
	4	Sistema actualiza stock	Sistema mostra confirmación e ticket ao cliente.
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Pago rexeitado	Sistema amosa erro e permite tentar outro método
	q	Erro de conexión	Sistema garda a compra como pendente e notifica despois
Rendimiento	A transacción debe procesarse en menos de 5 segundos.		
Frecuencia	Estímase unha media de 100 pagos diarios.		
Importancia	Vital		
Urgencia	Inmediata		
Comentarios	1. Soportar múltiples métodos (tarxeta, efectivo, móbil). 2. Cifrar todos os datos de pago. 3. Xerar comprobante dixital da compra. 4. Enviar confirmación por correo ao cliente. 5. Permitir ver historial de pagos realizados.		

CU08	**Xestionar produtos**		
Descripción	O administrador pode engadir, modificar ou eliminar produtos do catálogo da tenda, así como controlar o stock.		
Actores	Administrador		
Pre condiciones	1. O administrador iniciou sesión no panel de xestión. 2. O administrador ten permisos de xestión.		
Post condiciones	1. Os cambios gardanse na base de datos. 2. Os produtos están dispoñibles para a venda. 3. O stock actualízase correctamente.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Administrador accede á sección de produtos	Sistema mostra lista de produtos existentes
	2	Administrador selecciona acción (engadir/modificar/eliminar)	Sistema mostra formulario correspondente
	3	Administrador introduce datos do produto	Sistema valida os datos
			• 3.1 Se o código QR xa existe, o sistema amosa erro e non permite gardar
	4	Administrador garda os cambios	Sistema actualiza a base de datos
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Datos incompletos ou incorrectos	Sistema amosa erro e solicita corrección
	q	Erro ao gardar na base de datos	Sistema amosa aviso e permite reintentar
Rendimiento	A operación debe completarse en menos de 2 segundos.		
Frecuencia	Estímase unha media de 30 operacións diarias.		
Importancia	Alta		
Urgencia	Inmediata		
Comentarios	1. Permitir xerar código QR automático para novos productos 2. Validar que o prezo sexa maior que cero. 3. Non eliminar produtos con vendas asociadas, só desactivar. 4. Permitir importación masiva de produtos. 5. Mostrar historial de cambios realizados.		

CU09	Consultar vendas		
Descripción	O administrador pode consultar o historial de vendas realizadas na tenda, filtrando por data, cliente ou produto.		
Actores	Administrador		
Pre condiciones	1. O administrador iniciou sesión no panel. 2. Existen vendas rexistradas no sistema.		
Post condiciones	1. O administrador visualiza as vendas solicitadas. 2. Pode consultar o detalle de cada venda		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Administrador accede á sección de vendas	Sistema carga o historial de vendas
	2	Administrador aplica filtros (data, cliente, etc.)	Sistema filtra os resultados
	3	Sistema mostra lista de vendas	Sistema mostra detalles de cada venda
	4	Administrador selecciona unha venda para ver o detalle	Sistema mostra produtos, importe e método de pago
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Non hai vendas no período seleccionado	Sistema amosa mensaxe informando de que non hai resultados
	q	Erro ao cargar os datos	Sistema amosa erro e permite reintentar
Rendimiento	A consulta debe completarse en menos de 3 segundos.		
Frecuencia	Estímase unha media de 20 consultas diarias.		
Importancia	Alta		
Urgencia	Media		
Comentarios	1. Permitir exportar datos en formato CSV ou PDF. 2. Mostrar gráficos de ingresos por período. 3. Permitir buscar por número de venda. 4. Mostrar estado do pago de cada venda. 5. Permitir filtrar por método de pago.		

CU10	Pechar sesión		
Descripción	O usuario (Cliente ou Administrador) pecha a súa sesión no sistema de forma segura.		
Actores	Cliente, Administrador		
Pre condiciones	1. O usuario ten unha sesión activa.		
Post condiciones	1. A sesión péchase correctamente. 2. Elimínanse os datos de sesión do dispositivo. 3. O usuario debe autenticarse de novo para acceder.		
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Usuario selecciona "pechar sesión"	Sistema confirma a acción
	2	Usuario confirma que desexa pechar sesión	Sistema invalida o token de sesión
	3	Sistema pecha sesión	Sistema redirixe á pantalla de login
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Erro ao pechar sesión	Sistema intenta pechar de novo e notifica ao usuario
	q	O usuario cancela a operación	Sistema mantén a sesión activa
Rendimiento	O peche de sesión debe completarse en menos de 1 segundo.		
Frecuencia	Estímase unha media de 200 peches diarios.		
Importancia	Media		
Urgencia	Media		
Comentarios	1. Confirmar antes de pechar para evitar erros accidentais 2. Limpiar datos sensibles do dispositivo 3. Manter o carrito gardado para a próxima sesión (clientes) 4. Rexistrar a hora de peche de sesión 5. Pechar automáticamente tras período de inactividade		

CU11	Validar saída
Descripción	O cliente realiza unha validación final da súa compra nun punto de control (báscula ou operario) para obter a autorización de saída do local
Actores	Cliente, Sistema Externo (Báscula/Operario)
Pre condiciones	1. O cliente finalizou a compra e realizou o pago correctamente (CU07). 2. O cliente dispón do resumo da compra ou ticket dixital no seu dispositivo móbil.
Post	O cliente obtén un código de autorización de saída (QR ou barras)

condiciones			
Secuencia Normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	O cliente acode ao punto de control de saída e presenta o resumo da compra (QR)	O Sistema Externo (báscula ou lector do operario) le a información da venta
	2	O cliente coloca os artigos na báscula (se existe) ou permite a inspección manual	O Sistema Externo valida que o peso ou cantidade coincida co rexistro de venta
	3	O cliente confirma a finalización da validación no punto de control	O Sistema Externo xera e envía o código de autorización de saída ao dispositivo do cliente
	4	O cliente utiliza o código de autorización no torno de saída	O torno autoriza o paso e finaliza o proceso físico de compra
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	p	Discrepancia na validación (peso ou unidades incorrectas)	O sistema bloquea a xeración do código e solicita a intervención dun operario para unha revisión manual
	q	Erro na lectura do código de resumo da compra	O sistema permite reintentar o escaneo ou introducir manualmente o identificador da venda
	r	Fallo técnico no torno de saída	O operario realiza a apertura manual do paso tras verificar a autorización de saída no móbil do cliente
Rendimiento	A validación e xeración do código de saída debe realizarse en menos de 3 segundos		
Frecuencia	Estímase unha validación por cada venda completada (unha media de 100 diarias)		
Importancia	Alta (Garantiza a seguridade do establecemento e a integridade do stock)		
Urgencia	Inmediata		
Comentarios	1. O código de autorización de saída terá unha validez temporal moi limitada (por exemplo: 5 minutos) para evitar usos fraudulentos. 2. En tendas sen báscula, o operario validará de forma aleatoria ou total o contido do carriño antes de habilitar o código de saída. 3. O sistema debe rexistrar que a saída foi validada para evitar que o mesmo ticket se use máis dunha vez.		

8.2 Universo de discurso da base de datos

Para deseñar a base de datos do proxecto ScanGo realízase unha descrición dos elementos que forman parte do sistema e das relacións entre eles. Este universo de discurso recolle a información que a aplicación debe almacenar e xestionar.

Descrición xeral do dominio

ScanGo xestiona o proceso de autocompra nunha tenda física. No sistema interveñen clientes que realizan compras, produtos dispoñibles para a venda e administradores que xestionan o catálogo e o stock. Toda a información rexístrase nunha base de datos central.

Elementos principais

Os clientes son os usuarios da aplicación móbil. Deben rexistrarse para acceder ao sistema e poden escanear produtos, engadilos ao carriño virtual e finalizar compras. Cada cliente ten un identificador único e un historial de compras.

Os administradores xestionan o sistema mediante un panel de control. Poden engadir e modificar produtos, actualizar o stock e consultar vendas. Acceden mediante credenciais propias.

Os produtos son os artigos dispoñibles para a venda. Cada produto inclúe código QR, nome, descrición, prezo e stock dispoñible. Poden clasificarse por categorías.

O carriño de compra virtual é unha estrutura temporal asociada a un cliente onde se almacenan os produtos escaneados e as súas cantidades. Calcula o importe total antes de finalizar a compra.

As vendas rexístranse cando un cliente completa unha compra. Inclúen data, importe total e os produtos adquiridos. Ao completarse, actualízase automaticamente o stock.

As categorías permiten organizar os produtos segundo o seu tipo para facilitar a súa xestión. Cada pago está asociado a unha venda e inclúe o método empregado e o estado da transacción.

Relacións entre elementos

- Un cliente pode realizar varias vendas, pero cada venda pertence a un único cliente.
- Unha venda inclúe varios produtos a través da entidade DETALLE_VENDA, e un produto pode aparecer en distintas vendas.
- Cada cliente pode ter varios carriños ao longo do tempo. Cada carriño contén varios produtos a través de DETALLE_CARRIÑO.
- Cada produto pertence a unha categoría.
- Cada venda ten asociado un único pago.

Regras de xestión

- Non se poden realizar compras con stock insuficiente.
- O prezo dos produtos debe ser maior que cero.
- Os clientes deben estar autenticados para mercar.
- O stock actualízase automaticamente tras cada venda completada.
- Non se poden eliminar produtos con vendas rexistradas; unicamente desactivar.

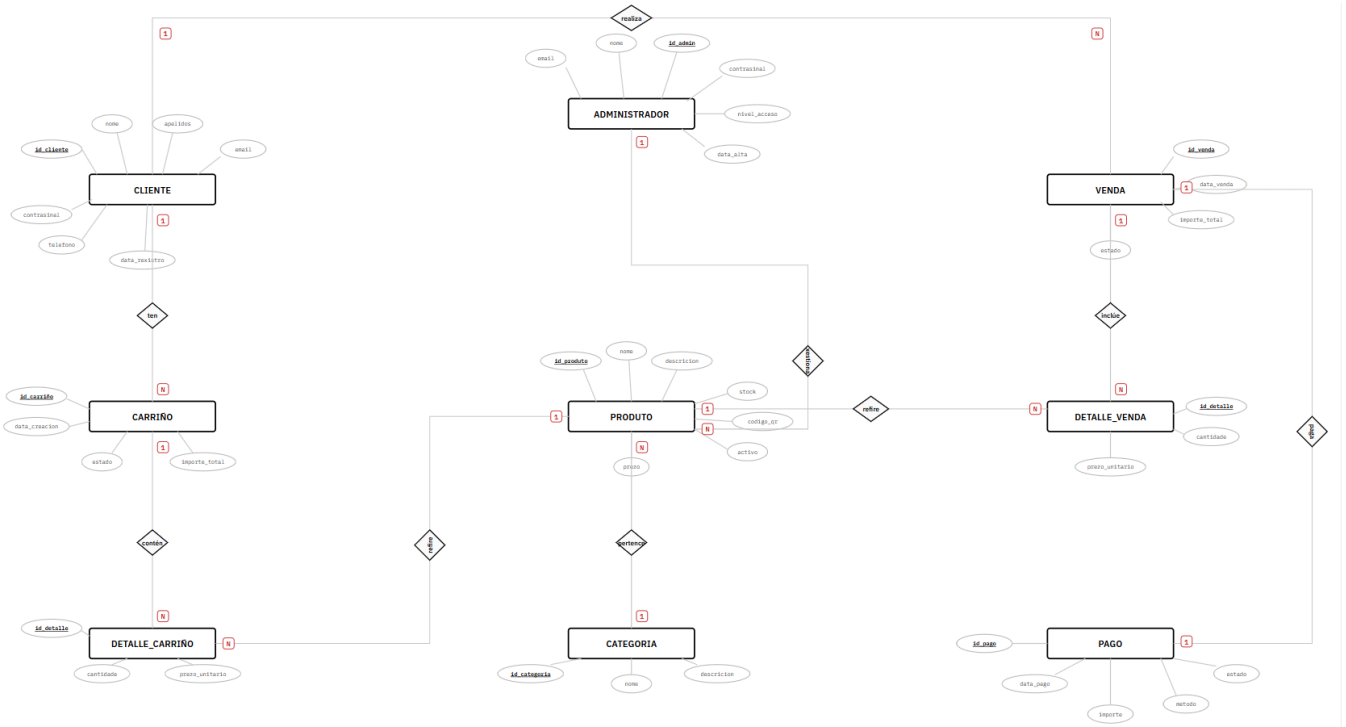
8.3 Diagrama Entidade-Relación

A partir do universo de discurso definido anteriormente, elaborouse o diagrama entidade-relación da base de datos de ScanGo seguindo a notación Chen. Este diagrama representa as entidades do sistema, os seus atributos principais e as relacións existentes entre elas, permitindo definir a estrutura lóxica da base de datos.

As entidades identificadas son: CLIENTE, ADMINISTRADOR, PRODUTO, CATEGORIA, CARRIÑO, DETALLE_CARRIÑO, VENDA, DETALLE_VENDA e PAGO. As entidades DETALLE_CARRIÑO e DETALLE_VENDA son entidades intermedias que resolven as relacións de moitos a moitos entre carriños e produtos, e entre vendas e produtos respectivamente.

As relacións e cardinalidades principais son as seguintes: un cliente pode realizar varias vendas (1:N); un cliente pode ter varios carriños ao longo do tempo (1:N); un carriño contén varios produtos a través de DETALLE_CARRIÑO (1:N); unha venda inclúe varios produtos a través de DETALLE_VENDA (1:N); varios produtos pertencen a unha mesma categoría (N:1); e cada venda ten asociado un único pago (1:1).

O deseño seguiu criterios de normalización, evitando redundancias mediante o uso de táboas intermedias. O prezo unitario gárdase nos detalles de venda para conservar o valor real dos produtos no momento da compra, garantindo a integridade do historial de transaccións.



9. Deseño do proxecto

9.1 Modelo relacional da base de datos

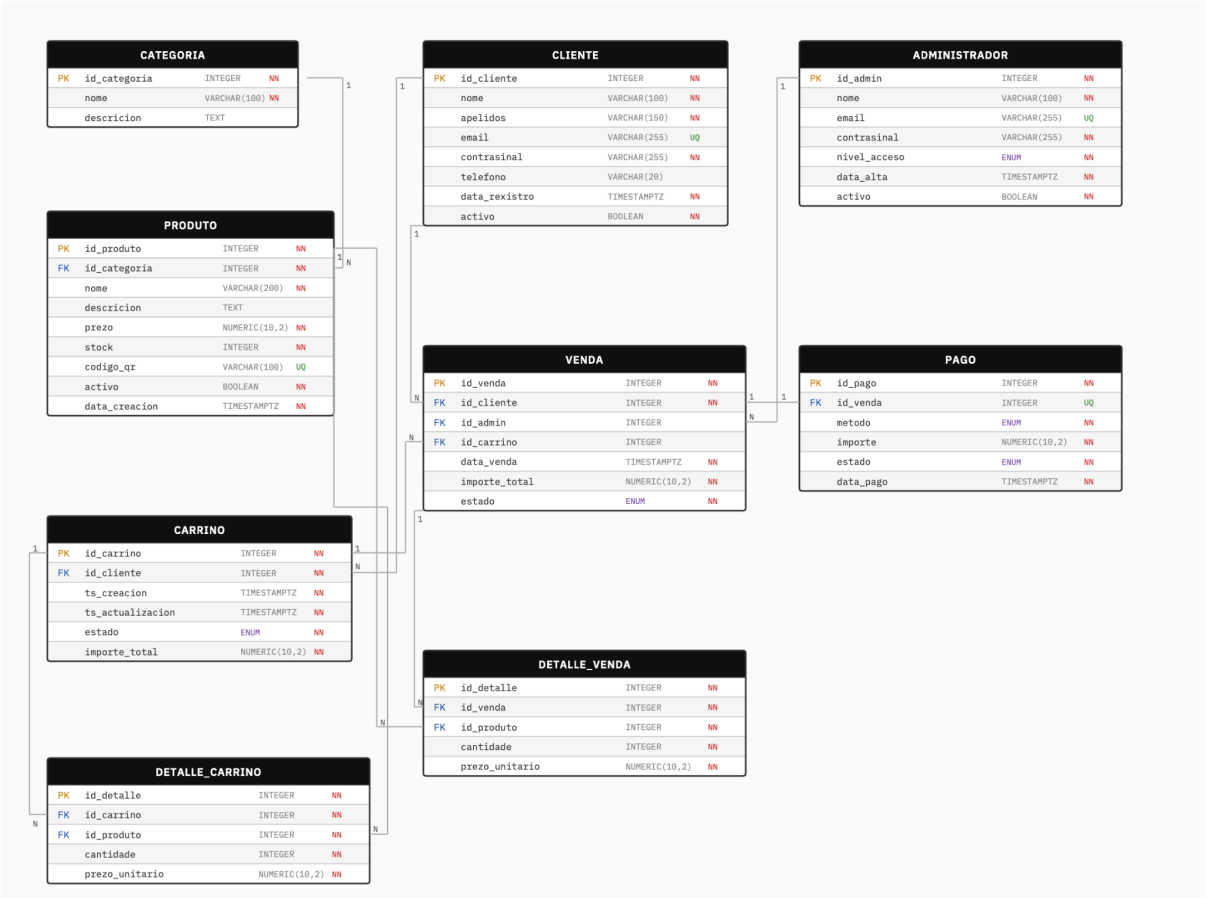
O modelo relacional derívase directamente do diagrama entidade-relación, materializando cada entidade nunha táboa e resolvendo as relacións mediante claves foráneas. As relacións de moitos a moitos representadas no E-R mediante entidades intermedias tradúcense en táboas con claves primarias propias e as correspondentes claves foráneas.

O modelo implémentase en PostgreSQL 15 e incorpora tipos enumerados (ENUM) para os campos de estado e método de pago, garantindo a consistencia dos valores almacenados. As restricións de integridade referencial defínense mediante cláusulas ON DELETE RESTRICT nos campos críticos e ON DELETE CASCADE nos detalles dependentes.

As táboas principais e os seus atributos son os seguintes:

- CLIENTE (id_cliente PK, nome, apelidos, email UQ, contrasinal, telefono, data_rexistro, activo)
- ADMINISTRADOR (id_admin PK, nome, email UQ, contrasinal, nivel_acceso, data_alta, activo)
- CATEGORIA (id_categoria PK, nome UQ, descricion)
- PRODUTO (id_produto PK, id_categoria FK, nome, descricion, prezo, stock, codigo_qr UQ, activo, data_creacion)
- CARRIÑO (id_carrino PK, id_cliente FK, ts_creacion, ts_actualizacion, estado, importe_total)
- DETALLE_CARRIÑO (id_detalle PK, id_carrino FK, id_produto FK, cantidade, prezo_unitario)
- VENDA (id_venda PK, id_cliente FK, id_admin FK, id_carrino FK, data_venda, importe_total, estado)
- DETALLE_VENDA (id_detalle PK, id_venda FK, id_produto FK, cantidade, prezo_unitario)
- PAGO (id_pago PK, id_venda FK UQ, metodo, importe, estado, data_pago)

O modelo inclúe dous triggers para automatizar a lóxica de negocio: o primeiro (trg_importe_carrino) actualiza automaticamente o importe total do carriño tras calquera modificación nos seus detalles; o segundo (trg_descontar_stock) desconta o stock dos produtos ao completarse un pago.



9.2 Diagrama de clases

O diagrama de clases representa a estrutura estática do sistema desde a perspectiva da programación orientada a obxectos, mostrando as clases que compoñen a aplicación, os seus atributos, os métodos asociados e as relacións existentes entre elas.

As clases definidas correspóndense coas entidades do modelo relacional, incorporando ademais os métodos necesarios para implementar a lóxica de negocio asociada aos casos de uso.

As relacións de composición reflicten dependencias estruturais fortes: Cliente compón Carriño, Carriño compón DetalleCarriño, e Venda compón DetalleVenda. Estas relacións indican que certos obxectos non poden existir de maneira independente. As relacións de asociación conectan Administrador con Venda, Categoria con Produto e Venda con Pago (1:1). As dependencias de DetalleCarriño e DetalleVenda cara a Produto represéntanse como relacións de dependencia descontinuas.

9.3 Diagramas de secuencia de cada caso de uso

Os diagramas de secuencia representan o comportamento dinámico do sistema, mostrando a orde temporal das interaccións entre os participantes para cada caso de uso. Os participantes definidos son o Cliente ou Usuario, a Aplicación móbil, o Sistema, a Base de datos e, no caso do pago, a Pasarela de pago como sistema externo.

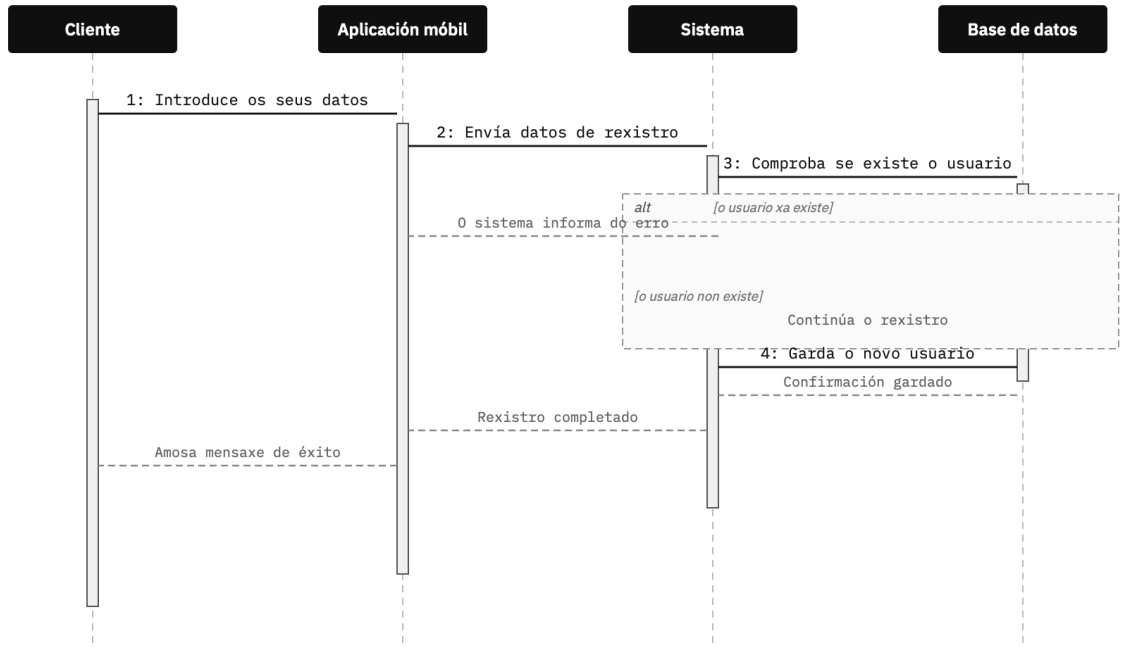
Os diagramas empregan a notación UML estándar: frechas sólidas para mensaxes síncronas, frechas descontinuas para respostas e retornos, barras de activación para indicar o período de execución de cada participante, e fragmentos combinados alt para representar fluxos alternativos e excepcións.

Para cada caso de uso represéntase o fluxo principal e as alternativas máis relevantes:

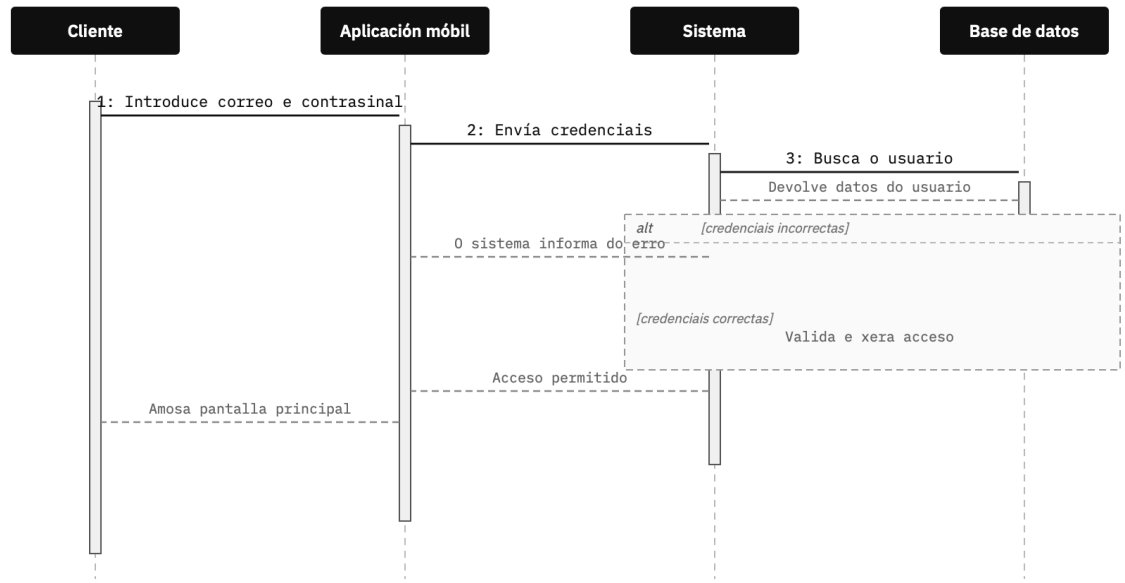
- CU01 – Rexistro de usuario: fluxo de rexistro con verificación de usuario existente.
- CU02 – Inicio de sesión: validación de credenciais con tratamento de erro.
- CU03 – Escanear produto: busca por código QR con tratamento de produto non atopado.

- CU04 – Engadir produto ao carriño: verificación de stock antes de engadir.
- CU05 – Ver carriño: carga de produtos e cálculo do importe total.
- CU06 – Finalizar compra: verificación de stock e xeración do rexistro de venda.
- CU07 – Realizar pago: procesamento mediante pasarela externa con tratamento de pago rexeitado.
- CU08 – Xestionar produtos: fluxo do administrador con tres alternativas (engadir, modificar, desactivar).
- CU09 – Consultar vendas: consulta filtrada con tratamento de lista baleira.
- CU10 – Pechar sesión: invalidación da sesión con tratamento de erro.

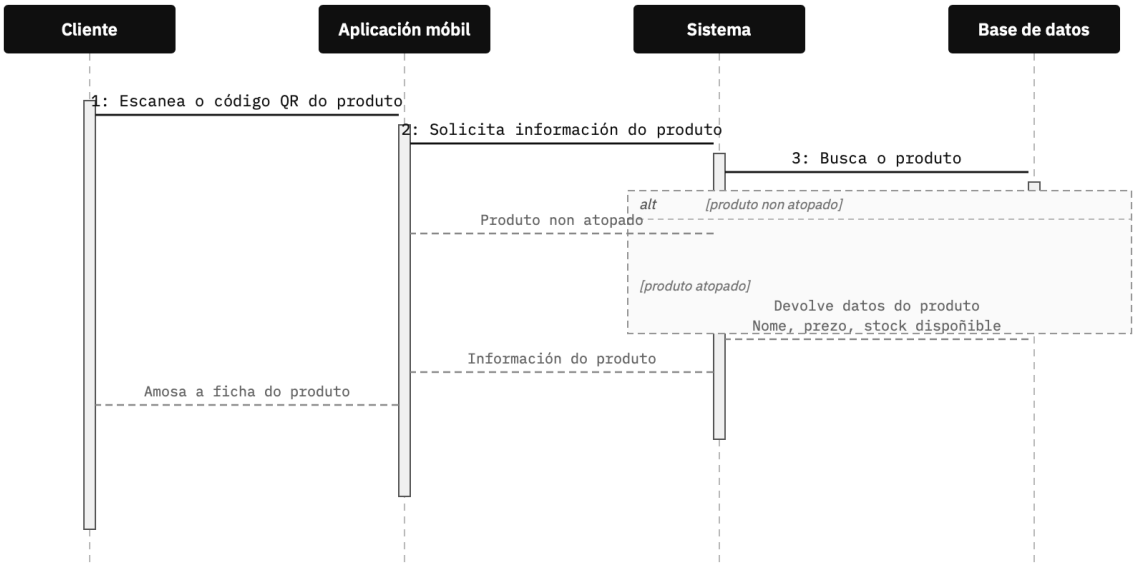
CU01 – Rexistro de usuario



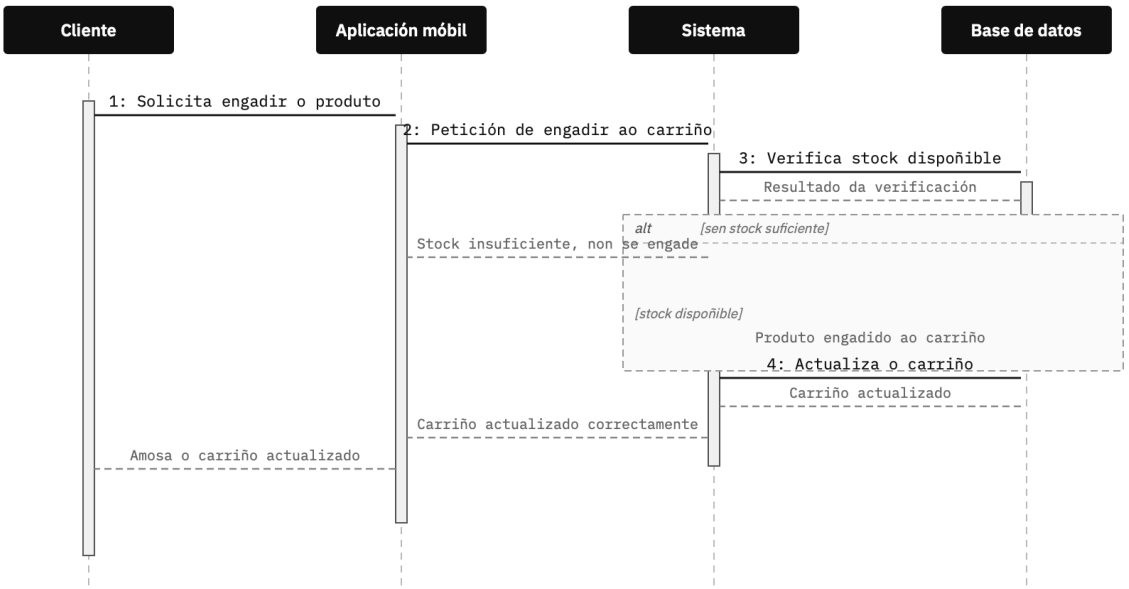
CU02 – Inicio de sesión



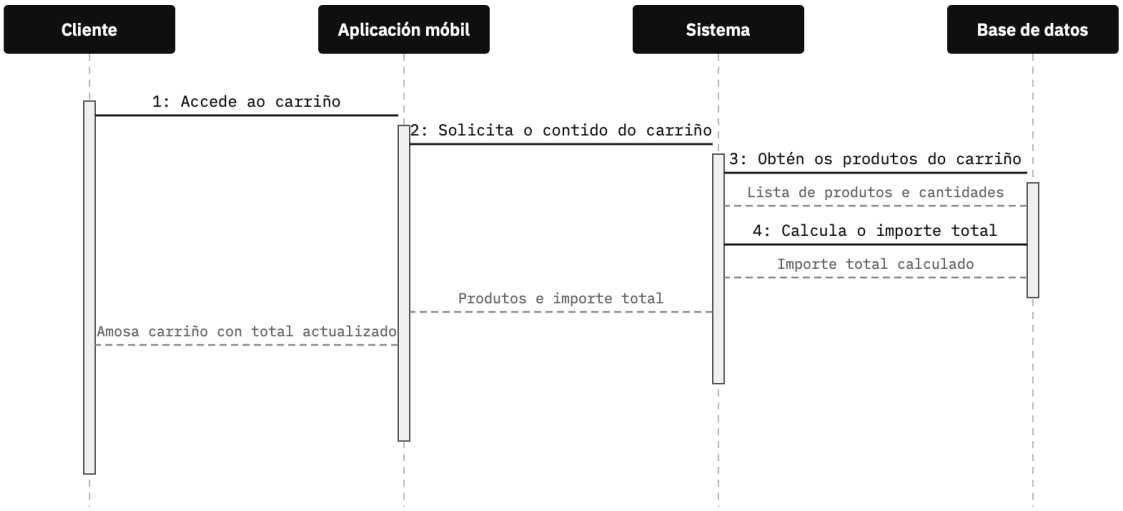
CU03 – Escanear produto



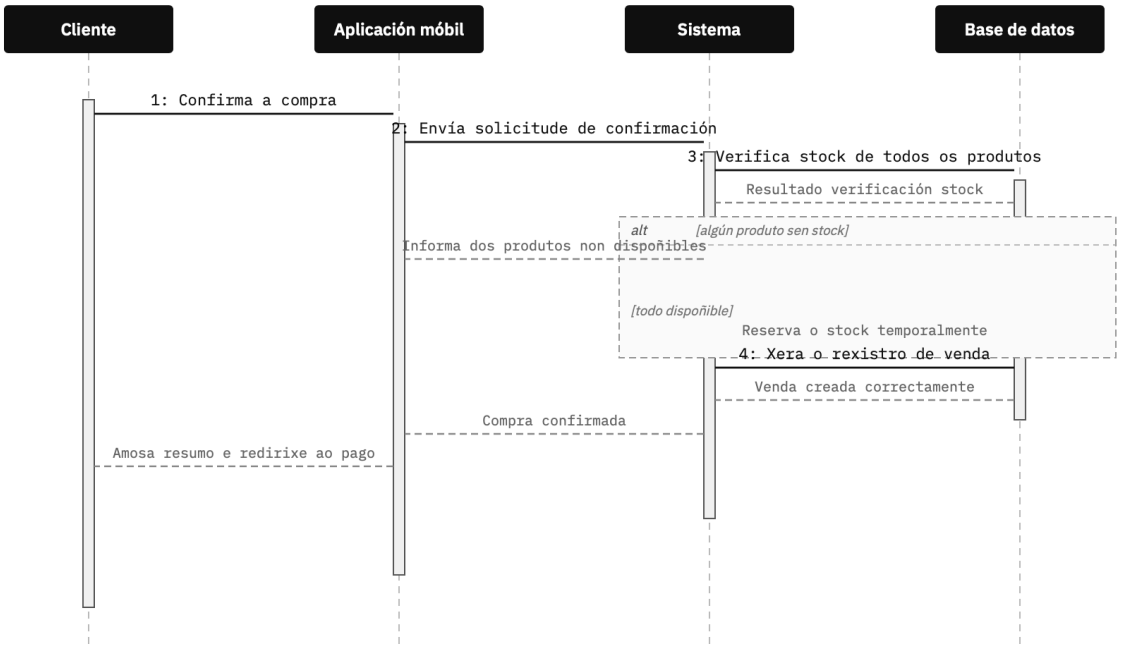
CU04 – Engadir produto ao carriño



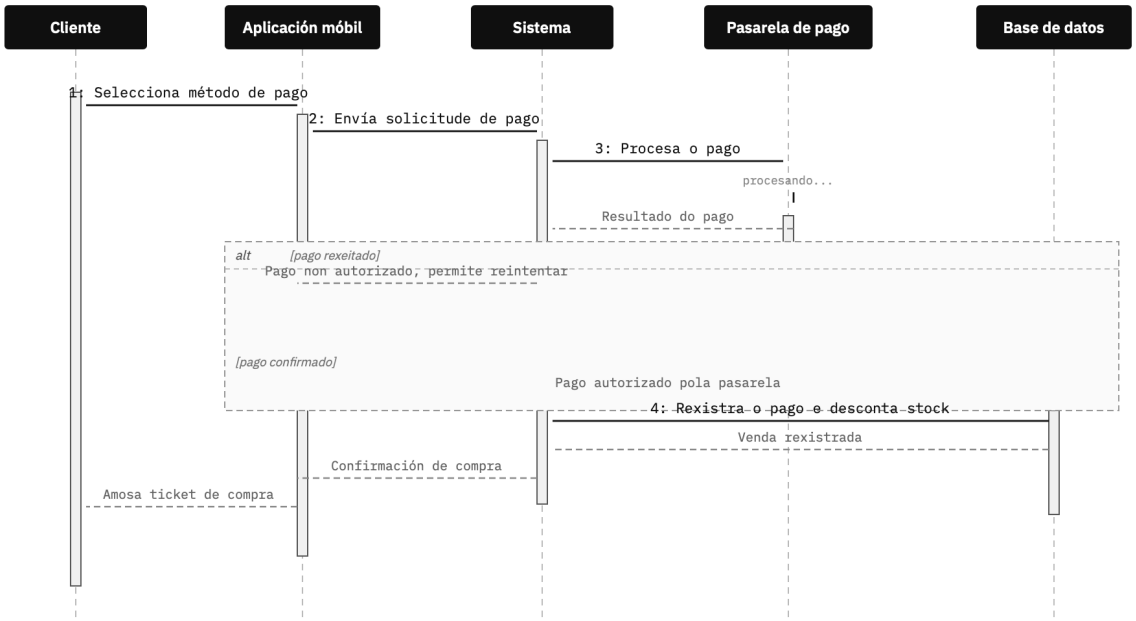
CU05 – Ver carriño



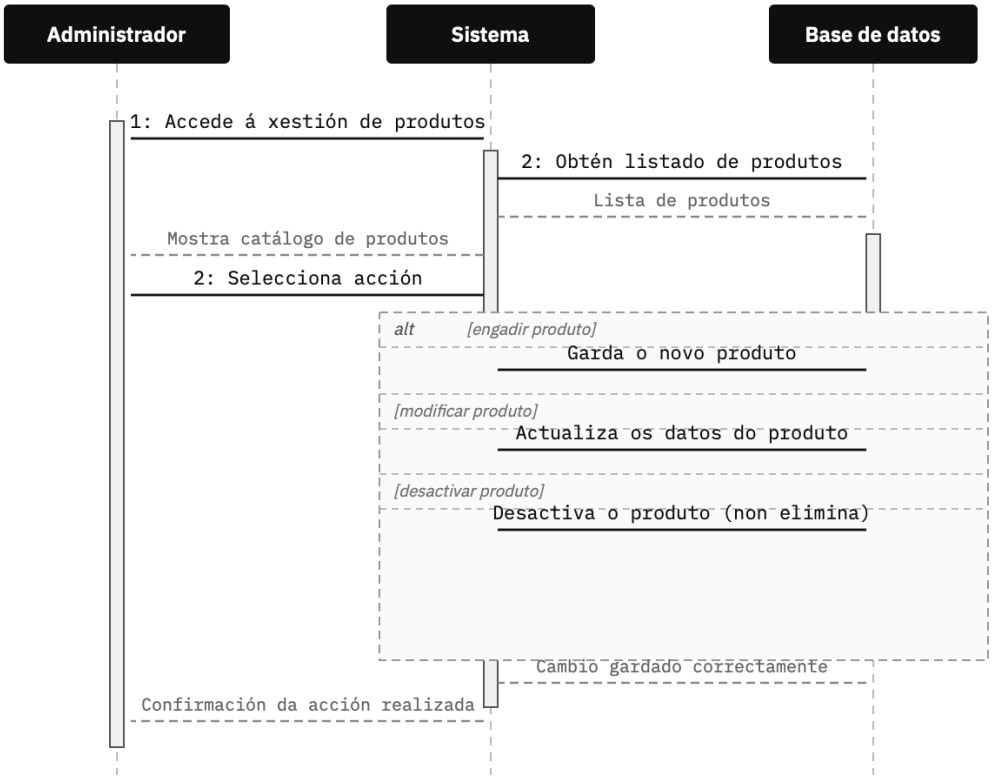
CU06 – Finalizar compra



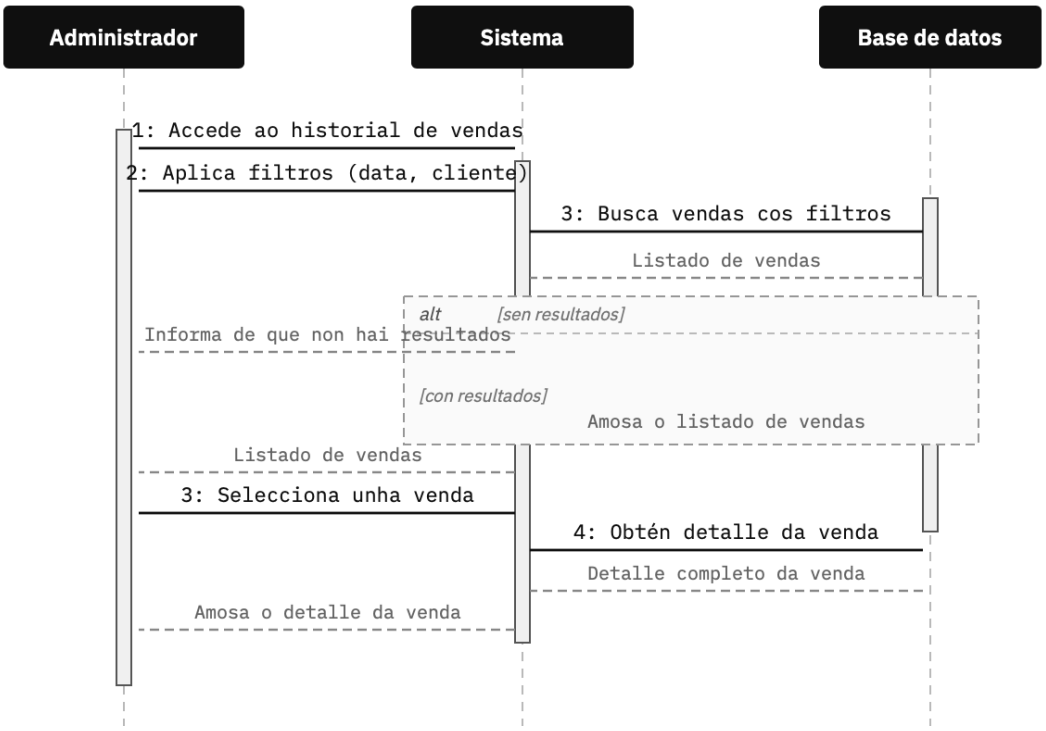
CU07 – Realizar pago



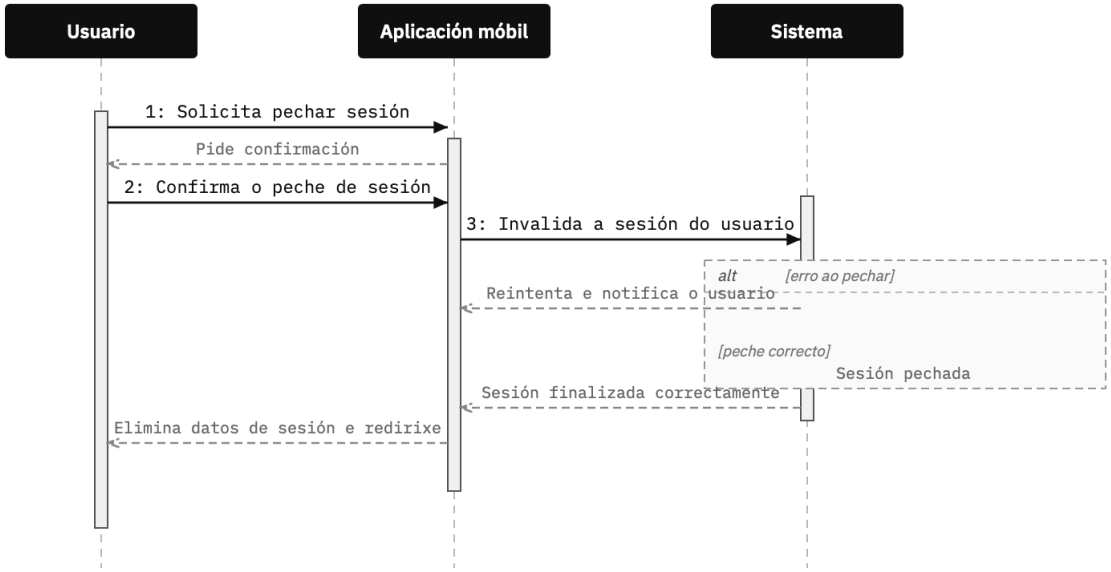
CU08 – Xestionar produtos (Administrador)

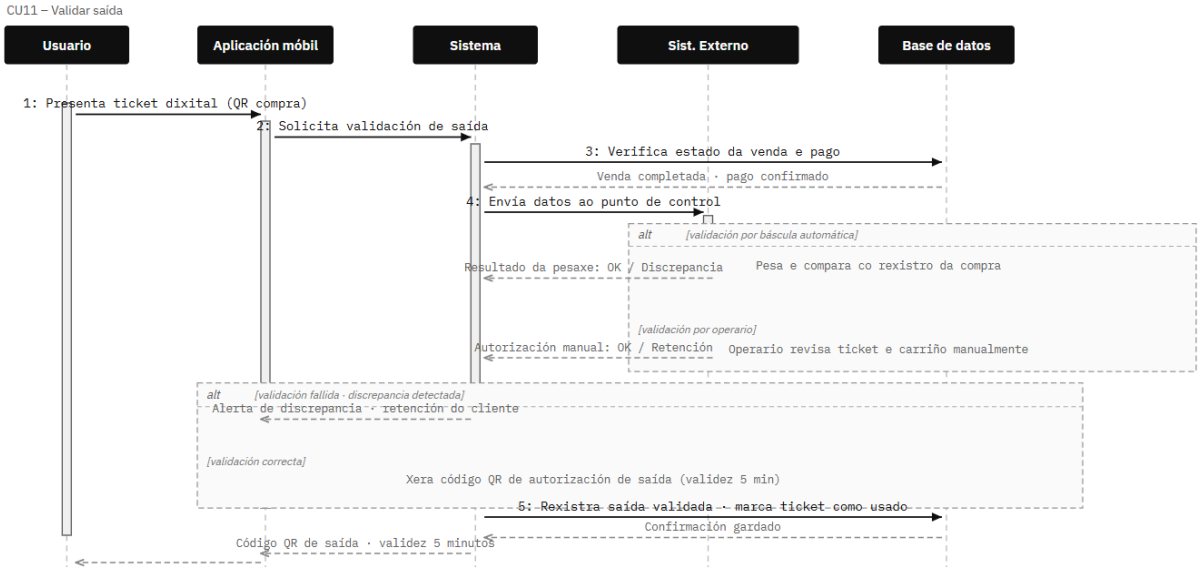


CU09 – Consultar vendas (Administrador)



CU10 – Pechar sesión





9.4 Diagrama de despregamento

O diagrama de despregamento representa como está organizada a infraestrutura do sistema ScanGo a nivel físico. Nel explícase en que dispositivos se executa cada parte da aplicación e como se comunican entre si mediante conexións seguras.

9.4.1 Nodos e contornas de execución

A arquitectura do sistema baséase nun modelo distribuído con tres partes principais:

- **Nodo cliente (smartphone)**
  - Corresponde ao móbil do usuario (Android ou iOS).
  - Executa a aplicación desenvolvida en Flutter, que se encarga da interface.
  - Permite escanear códigos QR usando a cámara do dispositivo.
  - Garda información do carriño de forma local con SQLite, polo que pode funcionar incluso sen conexión.
  - Comunícase coa API mediante peticións HTTPS.
- **Nodo servidor (Render.com)**
  - É a parte central do sistema, onde está a lóxica principal.
  - A API está feita con FastAPI en Python.
  - Encárgase de xestionar produtos, stock, sesións e datos dos usuarios.
  - A base de datos é PostgreSQL, onde se almacenan todos os datos do sistema.
  - O despregamento faise automaticamente mediante GitHub (cada cambio actualiza o servidor).
  - Inclúe medidas de seguridade como CORS, limitación de peticións e validación de datos.
- **Nodo externo (pasarela de pago)**
  - Servizo externo para xestionar pagos.
  - As operacións fanse de forma segura e cifrada.
  - Non se gardan datos de tarxetas, só tokens de sesión.

9.4.2 Protocolos e comunicacións

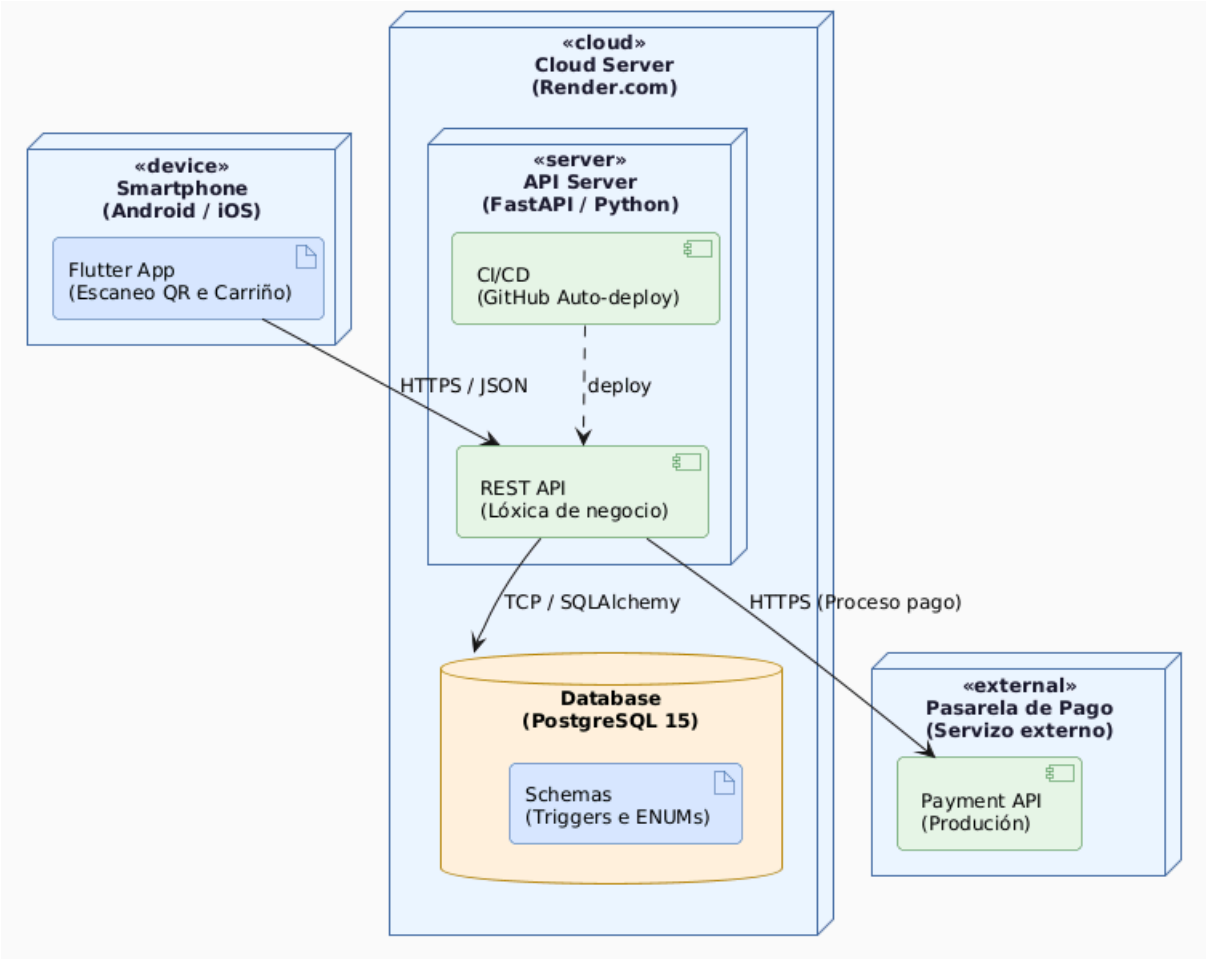
Para a comunicación entre as distintas partes do sistema úsanse os seguintes protocolos:

1. **HTTPS / JSON**
    - Todas as peticións entre o móbil e a API están cifradas.
    - Os datos envíanse en formato JSON.
2. **Conexión interna (SQLAlchemy)**
    - A API comunícase coa base de datos mediante SQLAlchemy.
    - Utilízanse conexións optimizadas para mellorar o rendemento.
3. **Despregamento automático**
    - GitHub conéctase con Render mediante webhooks.
    - Cada vez que se fai un cambio, o sistema actualízase automaticamente.
4. **SQLite no móbil**
    - Permite gardar datos de forma local.
    - Despois sincronízase coa base de datos principal mediante a API.

9.4.3 Seguridade

O sistema inclúe varias medidas de seguridade:

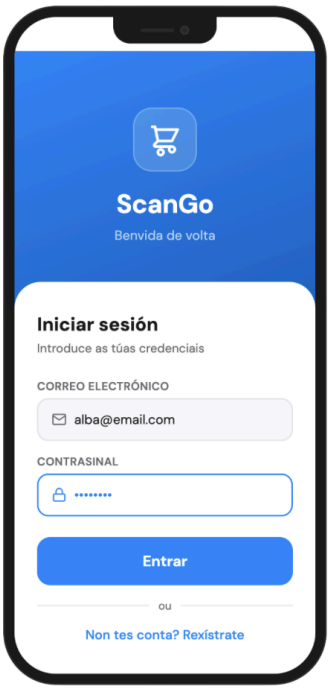
- Autenticación con tokens (JWT)
- Comunicación cifrada (HTTPS)
- Contrasinais encriptados
- Validación de datos na API
- Restrición de accesos (CORS)
- Limitación de peticións para evitar abusos



10.Mockups da interface.

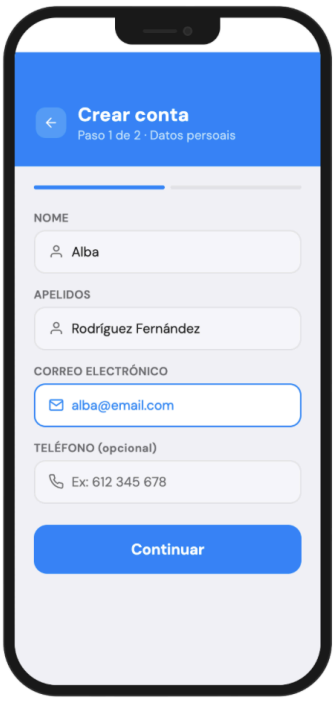
INICIO DE SESIÓN

Pantalla de acceso para clientes rexistrados. Inclúe validación de credenciais e acceso rápido ao rexistro.



REXISTRO DE USUARIO

Formulario de alta para novos clientes. Proceso en dous pasos con indicador de progreso.

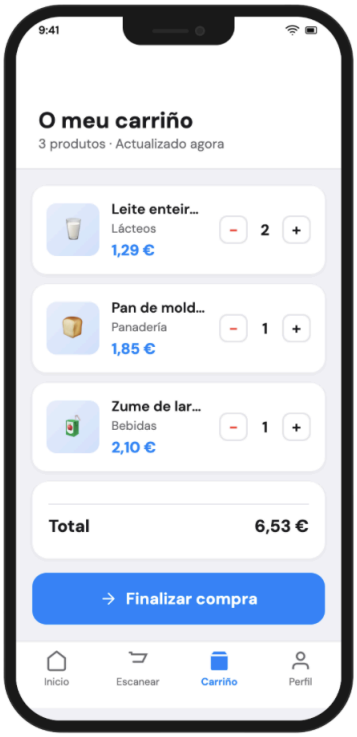


ESCANEAR PRODUTO

Cámara activa para ler o código QR do produto. Mostra a ficha do produto detectado na parte inferior.

CARRIÑO DE COMPRA

Listado dos produtos escaneados con cantidades, prezos unitarios e importe total da compra.



REALIZAR PAGO

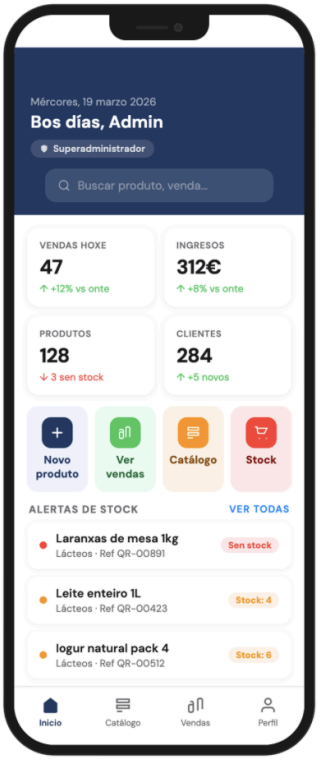
Selección do método de pago. Soporta tarxeta, efectivo e pago por móbil. Procesamento mediante pasarela externa.

COMPRA CONFIRMADA

Ticket dixital da compra completada. Mostra o resumo dos produtos, o importe e os datos da transacción.

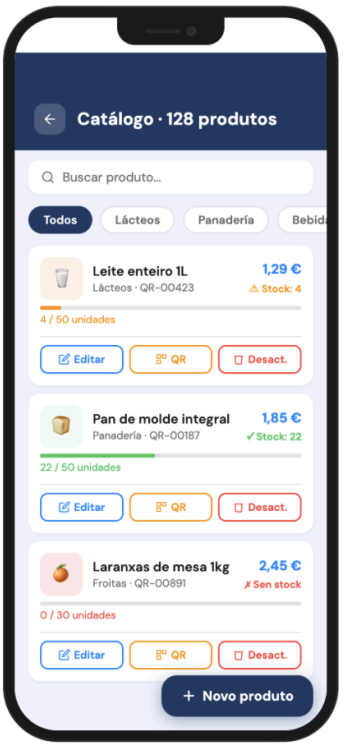
INICIO DO PANEL

Vista principal con estatísticas do día, accesos rápidos ás funcionalidades e alertas de stock baixo.



CATÁLOGO DE PRODUTOS

Listado completo do catálogo con filtros por categoría, barra de stock visual e accións rápidas por produto.



ENGADIR PRODUTO

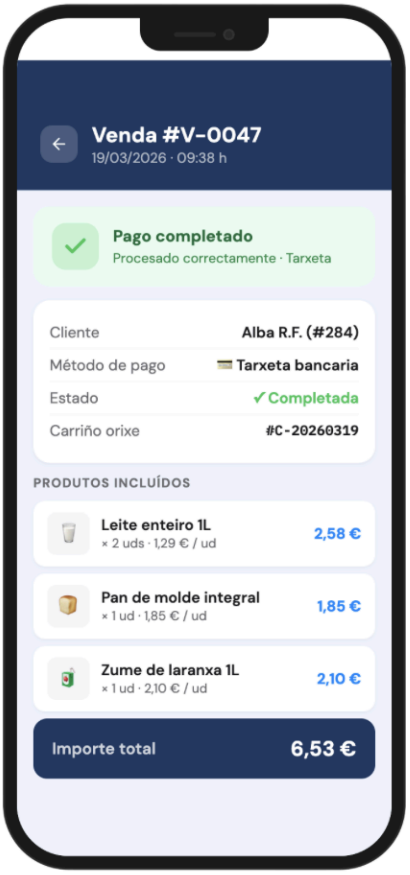
Formulario de alta de produto con todos os campos, selector de categoría, subida de imaxe e xeración automática de código QR.

HISTORIAL DE VENDAS

Listado de todas as vendas con filtros por data e estado, estatísticas resumidas e acceso ao detalle de cada transacción.

DETALLE DE VENDA

Vista completa dunha transacción: estado, datos do cliente, método de pago, produtos incluídos e importe total.



11. Especificación de alcance do sistema

Neste apartado explícase o estado actual do proxecto ScanGo, indicando que funcionalidades están xa feitas e cales quedan para máis adiante. O desenvolvemento seguiu unha metodoloxía áxil, polo que nesta fase centrámonos principalmente en asegurar o funcionamento do fluxo básico do sistema.

O obxectivo foi validar que a aplicación é capaz de escanear produtos, obter a súa información desde o backend e gardalos correctamente no carriño, garantindo unha comunicación fiable entre a aplicación móbil e o servidor.

11.1 Funcionalidades integradas na fase actual

Para comprobar o funcionamento do sistema, implementáronse as funcionalidades principais relacionadas co proceso de autocompra:

CU	Denominación	Descripción
CU03	Escanear produto	Uso da cámara do dispositivo para detectar códigos QR. O proceso execútase de forma asíncrona e realiza unha chamada á API para obter a información do produto en tempo real.
CU04	Engadir ao carriño	Permite engadir produtos ao carriño, gardalos tanto en local (SQLite) como no servidor e comprobar o stock dispoñible antes de engadilos.

Estas funcionalidades permiten realizar o fluxo básico da aplicación:

O usuario escanea un produto físico mediante código QR. A aplicación envía esa información á API, que procesa a petición e devolve os datos do produto almacenados na base de datos. Unha vez recibido, o usuario pode engadilo ao carriño, onde se garda a información e se actualiza o total da compra.

Deste xeito conséguese unha primeira versión funcional do sistema, que permite pasar dun produto físico a un rexistro dixital dentro da aplicación.

11.2 Funcionalidades diferidas (Iteracións posteriores)

Seguindo a metodoloxía áxil adoptada , o desenvolvemento centrouse na consolidación do fluxo crítico de captura de datos (escaneo QR) e a súa persistencia no servidor, garantindo a integridade da comunicación entre a aplicación móbil e o backend.

A. Autenticación baseada en JWT (CU01, CU02, CU10)

Aínda que actualmente se utiliza un usuario de proba, o sistema está preparado para implementar unha seguridade completa:

- **Seguridade con JWT:** Implementarase un sistema de autenticación mediante JSON Web Tokens (JWT) para garantir comunicacións seguras e sen estado entre a App Flutter e a API de FastAPI.
- **Cifrado:** Os contrasinais de clientes e administradores protexeranse na base de datos mediante o algoritmo de hash bcrypt, como se define no esquema SQL.
- **Xestión de sesións:** O peche de sesión (CU10) invalidará o token no dispositivo, eliminando calquera dato sensible de forma segura.

B. Persistencia e Edición do Carriño (CU05)

Para mellorar a experiencia de usuario en compras reais:

- **Sincronización:** Mellorase a integración entre a base de datos local (SQLite) e o servidor para que o cliente poida recuperar o seu carriño desde calquera dispositivo.
- **Edición avanzada:** Engadiranse funcións para modificar cantidades ou eliminar produtos directamente desde o resumo do carriño, con actualización instantánea do importe total.

C. Proceso de Pago e Venda (CU06, CU07)

A conversión do carriño virtual nunha transacción legal incluirá:

- **Reserva de Stock:** Aplicarase unha reserva temporal de produtos ao finalizar a compra para evitar que outros usuarios esgoten o stock durante o proceso de pago.
- **Pasarela de Pago:** Integración coa API externa para procesar tarxetas de crédito e outros métodos de pago electrónicos de xeito cifrado.
- **Comprobante Dixital:** Xeración automática do ticket de compra no historial do usuario tras a confirmación exitosa da pasarela.

D. Xestión Administrativa (CU08, CU09)

O panel para a tenda completará a súa operatividade con:

- **Mantemento do Catálogo:** Ferramentas para que o administrador poida dar de alta, modificar ou desactivar produtos, incluíndo a xeración de novos códigos QR.
- **Historial e Estatísticas:** Consulta detallada de vendas con filtros por data e cliente para facilitar o peche de caixa e o control de stock.

E. Validación de Saída (CU11)

Para pechar o ciclo físico da compra e garantir a seguridade:

- **QR de Autorización:** Tras o pago, o sistema xerará un código de saída con validez temporal limitada.
- **Punto de Control:** Integración co sistema externo (báscula ou operario) que validará que os artigos retirados coinciden co rexistro da venda antes de franquear o paso.

11.3 Xustificación de decisións técnicas

Durante esta fase tomáronse algunhas decisións para facilitar o desenvolvemento:

- **Uso de SQLite en local**
  Permite gardar datos no dispositivo e realizar probas de forma rápida. No futuro empregarase PostgreSQL en produción.
- **Uso dun usuario de proba**
  Simplifica o desenvolvemento ao non ter que implementar aínda o sistema completo de autenticación.
- **Procesos asíncronos**
  O escaneo e as chamadas á API realízanse de forma asíncrona para evitar bloqueos na aplicación.
- **Uso de Flutter**
  Permite desenvolver unha única aplicación compatible con Android e iOS.
- **Uso de FastAPI**
  Facilita o desenvolvemento da API, ofrece bo rendemento e xera documentación automática.

11.4 Límites do sistema

ScanGo céntrase na xestión dixital do proceso de compra, pero non na validación física dos produtos.

A comprobación de que os produtos retirados coinciden cos rexistrados depende de sistemas externos como:

- Básculas de peso
- Control manual por parte de operarios

A aplicación pode xerar un código ou token que serve como comprobante da compra e que poderá ser validado por estes sistemas.

11.5 Funcionalidades fora do alcance nesta fase

Deixarase como finalizada a funcionalidade a nivel lóxico dentro do sistema, pero non se implementará nesta fase a pasarela de pago nin o proceso de validación e saída dos produtos da tenda.

Estes elementos quedan fóra do alcance do proxecto actual, xa que requiren a integración con servizos externos (como plataformas de pago) e con sistemas físicos (como básculas ou control de acceso), que non forman parte deste desenvolvemento.

11.5.1 Responsabilidades

Para definir claramente o funcionamento do sistema, establécese a seguinte distribución de responsabilidades:

- **ScanGo:**
  - Escanear produtos
  - Xestionar o carriño
  - Controlar o stock
  - Gardar e procesar os datos
  - Comunicación coa API
- **Sistemas externos:**
  - Validación física dos produtos
  - Detección de posibles erros ou fraudes
  - Autorización da saída final